

Python for Network Automation: Efficiency, Error Reduction, and Agility

Why Python?

Python makes network automation practical because it replaces repetitive, error-prone manual work with scripts that just run.

Efficiency

Instead of logging into 50 routers one by one, a script hits all of them in minutes. You also get structured output — no more copying text out of a terminal and pasting it into a spreadsheet.

Fewer Mistakes

People typo. Scripts don't. Plus you can build in checks — like “only add this config line if it's not already there” — so you don't break things by accident.

Agility

Need to change something fleet-wide? Update one file, rerun the script. Need to undo? Roll back to the last version in git. You're not stuck doing the same manual steps every time a requirement changes.

Example: Safe NTP Config Push

Say every router needs the same NTP server. Instead of logging into each one, this script checks if the line exists first. If it's already there, it skips it. If it's missing, it adds it. Run it twice and nothing breaks — that's idempotency in plain terms.

```
# check before you add — prevents duplicates and human error
from netmiko import ConnectHandler

device = {
    "device_type": "cisco_ios",
    "host": "192.168.1.1",
    "username": "admin",
    "password": "secret",
}

conn = ConnectHandler(**device)

ntp_line = "ntp server 10.0.0.1"
current = conn.send_command("show run | include ntp")

if ntp_line in current:
    print("already there, skipping")
else:
    conn.send_config_set([ntp_line])
```

```
print("added")

conn.disconnect()
```

That's it. One script, one change, no duplicates, no manual SSH sessions.

Code Walkthrough

Import Netmiko

```
from netmiko import ConnectHandler
```

This pulls in the library that handles SSH for network gear. It knows Cisco IOS syntax, how to enter enable mode, how to deal with `--more--` prompts — all the stuff you'd have to script manually with Paramiko.

Define the Device

```
device = {
    "device_type": "cisco_ios",
    "host": "192.168.1.1",
    "username": "admin",
    "password": "secret",
}
```

This dict tells Netmiko what kind of device it's talking to and how to log in. `device_type` is the key — without it, Netmiko wouldn't know whether to send `terminal length 0` or `screen-length 0 temporary` or something else.

Open the Connection

```
conn = ConnectHandler(**device)
```

The `**` unpacks the dict into keyword arguments. Netmiko dials in, handles authentication, and drops you at a prompt. One line replaces about ten lines of raw Paramiko.

Set Up What You Want to Add

```
ntp_line = "ntp server 10.0.0.1"
```

This is the config line you're targeting. Keeping it in a variable means you only have to change it in one place if the NTP server IP changes later.

Pull the Current Config

```
current = conn.send_command("show run | include ntp")
```

This runs a filtered show command and stores the output as a string. The `| include ntp` narrows it down so you're not parsing a massive config blob.

The Check

```
if ntp_line in current:
    print("already there, skipping")
```

This is the guard. It looks at the string `current` and asks: is `ntp server 10.0.0.1` already in there? If yes, do nothing. Just say you're skipping.

This prevents duplicates. Without this check, running the script twice would add the same line twice. Cisco IOS doesn't always reject duplicate NTP lines — some versions just stack them, which can cause weird behavior.

The Push

```
else:
    conn.send_config_set([ntp_line])
    print("added")
```

Only runs if the line is missing. `send_config_set` automatically enters config mode, sends your command, and exits. You don't have to type `configure terminal` or `end` yourself.

Clean Up

```
conn.disconnect()
```

Closes the SSH session. Always do this — leaving connections hanging can exhaust session limits on the device.

How It Prevents Human Error

What Could Go Wrong Manually	What the Script Does Instead
You forget which routers you already configured	The script checks each one individually
You typo the NTP IP on router 37	The IP is in one variable, used everywhere
You add the line twice by accident	The <code>if</code> check blocks duplicates
You forget to exit config mode	<code>send_config_set</code> handles <code>end</code> automatically
You lose track of which devices you touched	The <code>print</code> statements log what happened

The big idea: The script doesn't assume anything. It looks before it leaps. That's idempotency — running it ten times gives the same result as running it once.

Sources

Liu, N.-Y. G., Triantis, K., Madsen, P., & Roets, B. (2025). The relationships among workload, automation reliance, and human errors in safety-critical monitoring roles. *Safety Science*, 185, 106775. <https://doi.org/10.1016/j.ssci.2024.106775>

Skjerve, A. B. M., & Skraaning, G. (2004). The quality of human-automation cooperation in human-system interface for nuclear power plants. *International Journal of Human-Computer Studies*, 61(5), 649–677. <https://doi.org/10.1016/j.ijhcs.2004.06.001>